# PYTHON CASES STUDIETS

## STUDIES CASE

1. Progressive Electricity Billing
2. Hospital billing and patient classification
3. Employee payroll and deduction analysis
4. Retail billing and with multiple discount rules
5. Manufacturing Cost and Profitability Analysis
6. Multi-Currency Travel Budget Analysis
7. Campus Event Financial Planning
8. Examination Result and Grace-Mark Validation
9. Attendance recovery computation
10. Advanced Numeric Pattern Generator

## 1.PROGRESSIVE ELECTRICITY BILLING

**ALGORITHM:**

**1.**Start

2.Read consumer name and units consumed.

3.If units are up to 100, charge ₹1.50 per unit; otherwise charge ₹1.50 for the first 100 units.

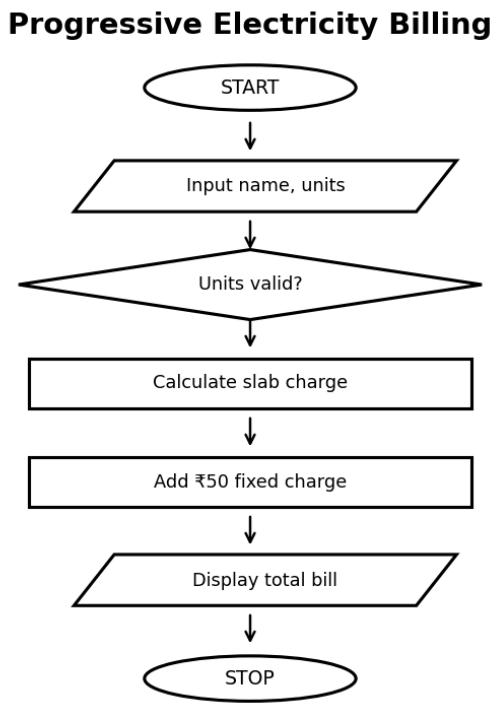
4.For units 101–200, charge ₹2.50 per unit in that slab; for units above 200, charge ₹4.00 per unit in that slab.

5.Add a fixed meter charge of ₹50.

6.Display units and total bill.

7.Stop.

**FLOWCHART:**



**PSEUDOCODE:**

START
INPUT name, units
IF units <= 100:
  energy_charge = units * 1.50
ELSE IF units <= 200:
  energy_charge = 100 * 1.50 + (units - 100) * 2.50
ELSE:
  energy_charge = 100 * 1.50 + 100 * 2.50 + (units - 200) * 4.00
total = energy_charge + 50
DISPLAY name, units, total
STOP

**CODE:**

```
units = int(input("Enter electricity units: "))
if units <= 100:
    bill = units * 2
elif units <= 200:
    bill = 200 + (units - 100) * 3
else:
    bill = 500 + (units - 200) * 5
print("Electricity Bill: Rs.", bill)
```

**OUTPUT:**

```
Enter electricity units: 3
Electricity Bill: Rs. 6
```

**TESTCASES:**

| Test Case | Input (Units) | Expected Output |
|---|---|---|
| TC1 | 50 | Electricity Bill: Rs. 100 |
| TC2 | 100 | Electricity Bill: Rs. 200 |
| TC3 | 150 | Electricity Bill: Rs. 350 |
| TC4 | 200 | Electricity Bill: Rs. 500 |
| TC5 | 250 | Electricity Bill: Rs. 750 |
| TC6 | 0 | Electricity Bill: Rs. 0 |

## 2.Hospital billing and patient classification

**ALGORITHM:**

1.Start.

2. Read patient name, patient type (general or private), days admitted, and treatment charge

3. patient name, patient type (general or private), days admitted, and treatment charge.

4.Validate days and treatment charge are non-negative.

5.Calculate room charge: ₹1,000 per day for private or ₹500 per day for general.

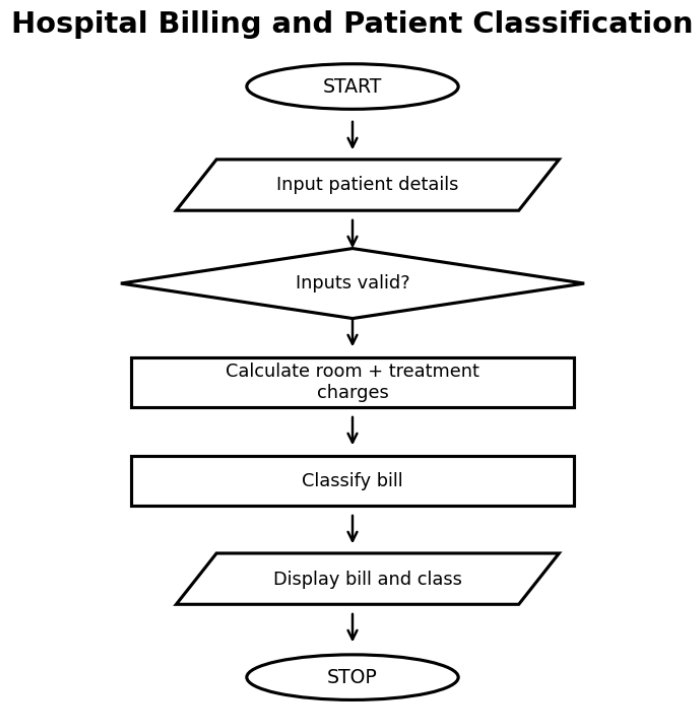
6.Add treatment charge to room charge.

7.Classify bill as Low (below ₹5,000), Medium (₹5,000–₹15,000), or High (above ₹15,000).

8.Display total and classification.

9.Stop.

**FLOWCHART:**




**PSEUDOCODE:**

START
INPUT name, patient_type, days, treatment
IF days < 0 OR treatment < 0: DISPLAY "Invalid input"
ELSE:
  IF patient_type == "private": room_rate = 1000
  ELSE: room_rate = 500
  total = days * room_rate + treatment
  IF total < 5000: category = "Low"
  ELSE IF total <= 15000: category = "Medium"
  ELSE: category = "High"
  DISPLAY name, total, category
STOP

**CODE:**

```python
name = input("Enter patient name: ")
age = int(input("Enter patient age: "))
if age < 18:
    print("Category: Child")
elif age < 60:
    print("Category: Adult")
else:
    print("Category: Senior Citizen")
print("1. General Checkup - Rs. 500")
print("2. Blood Test - Rs. 800")
print("3. X-Ray - Rs. 1000")
choice = int(input("Enter your choice: "))
if choice == 1:
    bill = 500
elif choice == 2:
    bill = 800
elif choice == 3:
    bill = 1000
else:
    bill = 0
    print("Invalid choice")
if bill > 0:
    print("Patient Name:", name)
    print("Hospital Bill: Rs.", bill)
```

**OUTPUT:**

```
Enter patient name: SARATHA
Enter patient age: 46
Category: Adult
1. General Checkup - Rs. 500
2. Blood Test - Rs. 800
3. X-Ray - Rs. 1000
Enter your choice: 80
Invalid choice
```

**TESTCASES:**

| Test Case | Patient Name | Age | Choice | Expected Output |
|---|---|---|---|---|
| TC1 | Ravi | 10 | 1 | Category: Child, Hospital Bill: Rs. 500 |
| TC2 | Anu | 25 | 2 | Category: Adult, Hospital Bill: Rs. 800 |
| TC3 | Kumar | 65 | 3 | Category: Senior Citizen, Hospital Bill: Rs. 1000 |
| TC4 | Priya | 17 | 3 | Category: Child, Hospital Bill: Rs. 1000 |

## 3.Employee payroll and deduction analysis

**ALGORITHM:**

1.Start.

2.Read employee name, basic salary, and tax deduction percentage.

3.Calculate HRA as 20% of basic salary and DA as 10%.

4.Calculate gross salary = basic + HRA + DA.

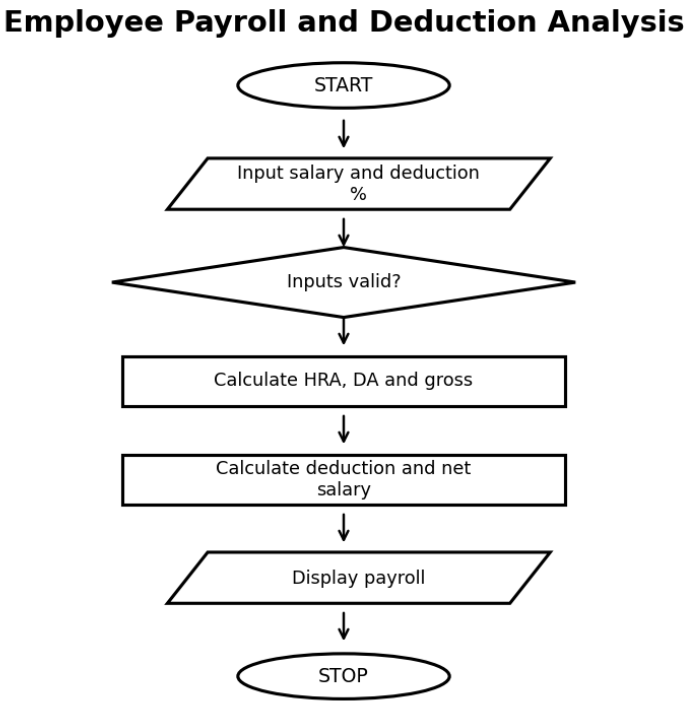
5.Calculate deduction using the given percentage of gross salary.

6.Calculate net salary = gross − deduction.

7.Display gross, deduction, and net salary.

8.Stop.

**FLOWCHART:**




**PSEUDOCODE:**

```
START
INPUT name, basic, deduction_percent
IF basic < 0 OR deduction_percent < 0 OR deduction_percent > 100:
   DISPLAY "Invalid input"
ELSE:
   hra = basic * 0.20
   da = basic * 0.10
   gross = basic + hra + da
   deduction = gross * deduction_percent / 100
   net = gross - deduction
   DISPLAY name, gross, deduction, net
STOP
```

**CODE:**

```
name = input("Enter employee name: ")
salary = float(input("Enter basic salary: "))
if salary <= 10000:
    deduction = salary * 0.05
elif salary <= 20000:
    deduction = salary * 0.10
else:
    deduction = salary * 0.15
net_salary = salary - deduction
print("Employee Name:", name)
print("Basic Salary: Rs.", salary)
print("Deduction: Rs.", deduction)
print("Net Salary: Rs.", net_salary)
```

**OUTPUT:**

```
Enter employee name: SUJATHA
Enter basic salary: 100000
Employee Name: SUJATHA
Basic Salary: Rs. 100000.0
Deduction: Rs. 15000.0
Net Salary: Rs. 85000.0
```

**TESTCASES:**

| est Case | Employee Name | Basic Salary (Input) | Deduction (Output) | Net Salary (Output) |
|---|---|---|---|---|
| TC1 | Ravi | 8000 | 400.0 | 7600.0 |
| TC2 | Sita | 10000 | 500.0 | 9500.0 |
| TC3 | Raju | 15000 | 1500.0 | 13500.0 |
| TC4 | Priya | 20000 | 2000.0 | 18000.0 |
| TC5 | Kiran | 25000 | 3750.0 | 21250.0 |
| TC6 | Arun | 5000 | 250.0 | 4750.0 |

## 4.Retail billing and with multiple discount rules

**ALGORITHM:**

1.Start.

2.Read item total and whether the customer is a member (yes/no).

3.If total is at least ₹5,000, apply 10% discount; otherwise if at least ₹2,000, apply 5%; otherwise no discount.

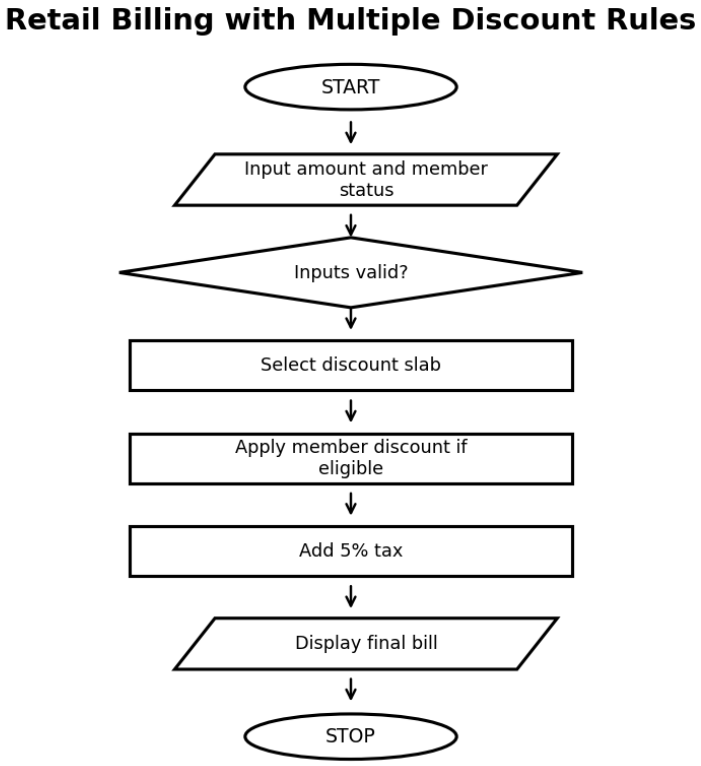
4.If customer is a member, apply an additional 2% discount to the amount after the first discount.

5.Add 5% tax to the discounted amount.

6.Display discount and final bill.

7.Stop.

**FLOWCHART:**




**PSEUDOCODE:**

START
INPUT amount, member
IF amount < 0: DISPLAY "Invalid amount"
ELSE:
  IF amount >= 5000: discount = amount * 0.10
  ELSE IF amount >= 2000: discount = amount * 0.05
  ELSE: discount = 0
  after_discount = amount - discount
  IF member == "yes": after_discount = after_discount * 0.98
  tax = after_discount * 0.05
  total = after_discount + tax
  DISPLAY discount, tax, total
STOP

**CODE:**

```python
amount = float(input("Enter shopping amount: "))
if amount >= 5000:
    discount = amount * 0.20
elif amount >= 2000:
    discount = amount * 0.10
elif amount >= 1000:
    discount = amount * 0.05
else:
    discount = 0
final_amount = amount - discount
print("Shopping Amount: Rs.", amount)
print("Discount: Rs.", discount)
print("Final Bill: Rs.", final_amount)
```

**OUTPUT:**

```
Enter shopping amount: 10000
Shopping Amount: Rs. 10000.0
Discount: Rs. 2000.0
Final Bill: Rs. 8000.0

Process finished with exit code 0
```

**TESTCASES:**

| Test Case | Shopping Amount (₹) | Discount Rate | Discount (₹) | Final Bill (₹) |
|---|---|---|---|---|
| TC1 | 500 | 0% | 0 | 500 |
| TC2 | 1000 | 5% | 50 | 950 |
| TC3 | 1500 | 5% | 75 | 1425 |
| TC4 | 2000 | 10% | 200 | 1800 |
| TC5 | 3000 | 10% | 300 | 2700 |
| TC6 | 5000 | 20% | 1000 | 4000 |
| TC7 | 8000 | 20% | 1600 | 6400 |

## 5.Manufacturing cost and profitability analysis

**ALGORITHM:**

1.Start.

2.Read quantity produced, material cost per unit, labour cost per unit, and selling price per unit.

3.Validate all values are non-negative.

4.Calculate total cost = quantity × (material cost + labour cost).

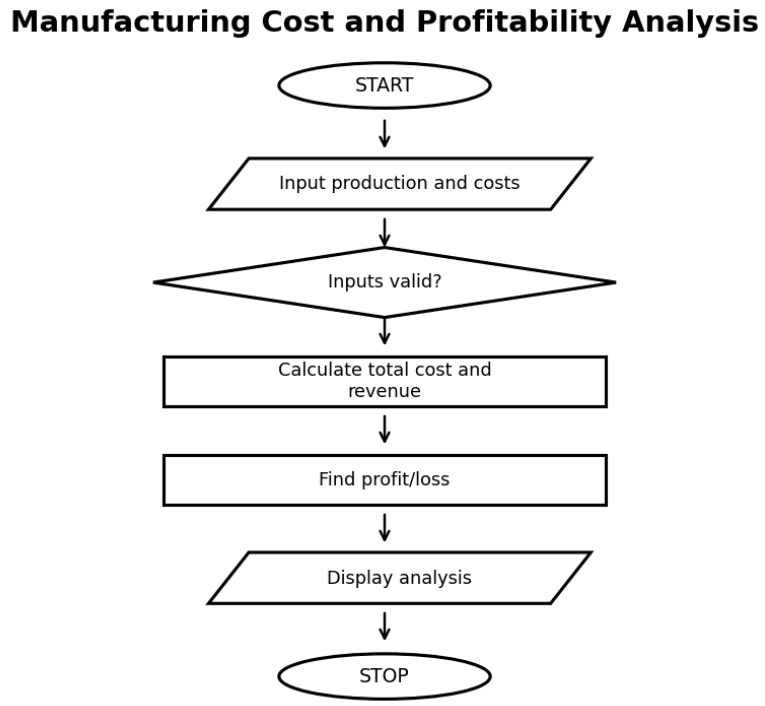
5.Calculate revenue = quantity × selling price.

6.Calculate profit or loss = revenue − total cost.

7.Display cost, revenue, and profit/loss.

8.Stop.

**FLOWCHART:**




**PSEUDOCODE:**

```
START
INPUT quantity, material, labour, selling_price
IF any input is negative: DISPLAY "Invalid input"
ELSE:
   total_cost = quantity * (material + labour)
   revenue = quantity * selling_price
   result = revenue - total_cost
   IF result > 0: status = "Profit"
   ELSE IF result < 0: status = "Loss"
   ELSE: status = "Break-even"
   DISPLAY total_cost, revenue, result, status
STOP
```

**CODE:**

```python
material = float(input("Enter material cost: "))
labour = float(input("Enter labour cost: "))
other = float(input("Enter other costs: "))
quantity = int(input("Enter number of products: "))
price = float(input("Enter selling price per product: "))
total_cost = material + labour + other
total_sales = quantity * price
profit = total_sales - total_cost
print("Total Manufacturing Cost: Rs.", total_cost)
print("Total Sales: Rs.", total_sales)
print("Profit/Loss: Rs.", profit)
if profit > 0:
    print("Profit")
elif profit < 0:
    print("Loss")
else:
    print("No Profit, No Loss")
```

**OUTPUT:**

```
Enter material cost: 100000
Enter labour cost: 200000
Enter other costs: 50000
Enter number of products: 6
Enter selling price per product: 10000
Total Manufacturing Cost: Rs. 350000.0
Total Sales: Rs. 60000.0
Profit/Loss: Rs. -290000.0
Loss
```

**TESTCASES:**

| Test Case | Material Cost (₹) | Labour Cost (₹) | Other Costs (₹) | Quantity | Selling Price (₹) | Total Cost (₹) | Total Sales (₹) | Profit/Loss (₹) | Result |
|---|---|---|---|---|---|---|---|---|---|
| TC1 | 5000 | 2000 | 1000 | 100 | 100 | 8000 | 10000 | 2000 | Profit |
| TC2 | 5000 | 2000 | 1000 | 100 | 80 | 8000 | 8000 | 0 | No Profit, No Loss |
| TC3 | 5000 | 2000 | 1000 | 100 | 60 | 8000 | 6000 | -2000 | Loss |
| TC4 | 3000 | 1000 | 500 | 50 | 100 | 4500 | 5000 | 500 | Profit |
| TC5 | 2000 | 1000 | 500 | 20 | 100 | 3500 | 2000 | -1500 | Loss |
| TC6 | 1000 | 500 | 500 | 10 | 200 | 2000 | 2000 | 0 | No Profit, No Loss |

## 6.Multi-currency travel budget analysis

**ALGORITHM:**

1.Start.

2.Read travel budget in local currency, exchange rate (foreign units per local unit), and estimated foreign expenses.

3.Validate inputs are non-negative.

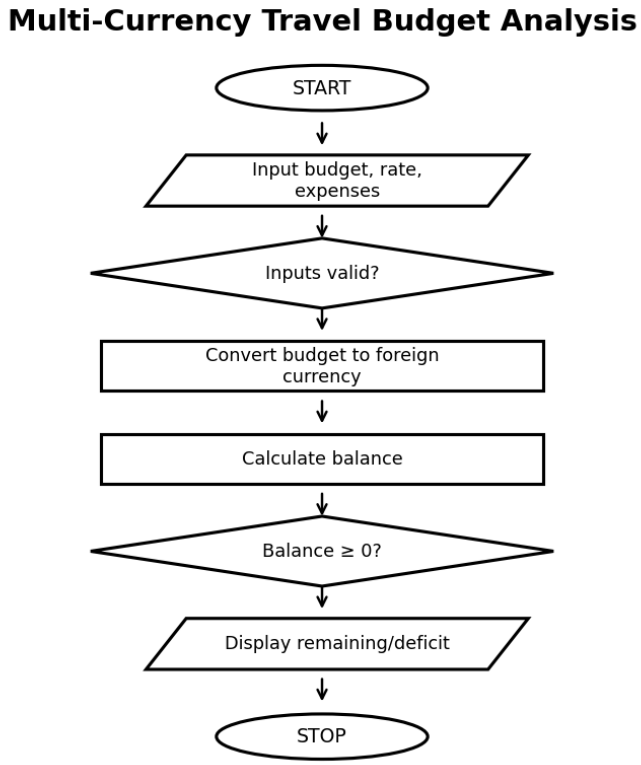
4.Convert budget to foreign currency using budget × exchange rate.

5.Calculate remaining balance = converted budget − expenses.

6.If balance is non-negative, display remaining balance; otherwise display the deficit.

7.Stop.

**FLOWCHART:**




**PSEUDOCODE:**

```
START
INPUT budget, rate, expenses
IF budget < 0 OR rate < 0 OR expenses < 0:
   DISPLAY "Invalid input"
ELSE:
   converted_budget = budget * rate
   balance = converted_budget - expenses
   IF balance >= 0: DISPLAY "Remaining budget", balance
   ELSE: DISPLAY "Budget exceeded by", abs(balance)
STOP
```

**CODE:**

```
budget = float(input("Enter travel budget in rupees: "))
currency = input("Enter currency (USD or EUR): ").upper()
amount = float(input("Enter foreign currency amount: "))
if currency == "USD":
    rate = 85
elif currency == "EUR":
    rate = 95
else:
    rate = 0
    print("Invalid currency")
if rate > 0:
    expense = amount * rate
    remaining = budget - expense
    print("Expense in Rupees: Rs.", expense)
    print("Remaining Budget: Rs.", remaining)
    if remaining >= 0:
        print("Budget is sufficient")
    else:
        print("Budget exceeded")
```

**OUTPUT:**

```
Enter travel budget in rupees: 100000
Enter currency (USD or EUR): usd
Enter foreign currency amount: 20000
Expense in Rupees: Rs. 1700000.0
Remaining Budget: Rs. -1600000.0
Budget exceeded
```

**TESTCASES:**

| Test Case | Budget (₹) | Currency | Foreign Amount | Rate (₹) | Expense (₹) | Remaining Budget (₹) | Expected Result |
|---|---|---|---|---|---|---|---|
| TC1 | 10000 | USD | 50 | 85 | 4250 | 5750 | Budget is sufficient |
| TC2 | 5000 | USD | 60 | 85 | 5100 | -100 | Budget exceeded |
| TC3 | 15000 | EUR | 100 | 95 | 9500 | 5500 | Budget is sufficient |
| TC4 | 8000 | EUR | 100 | 95 | 9500 | -1500 | Budget exceeded |
| TC5 | 5000 | USD | 0 | 85 | 0 | 5000 | Budget is sufficient |

## 7.Campus event financial planning

**ALGORITHM:**

1.Start.

2.Read expected attendees, ticket price, sponsorship, and fixed event cost.

3.Calculate ticket income = attendees × ticket price.

4.Calculate total income = ticket income + sponsorship.

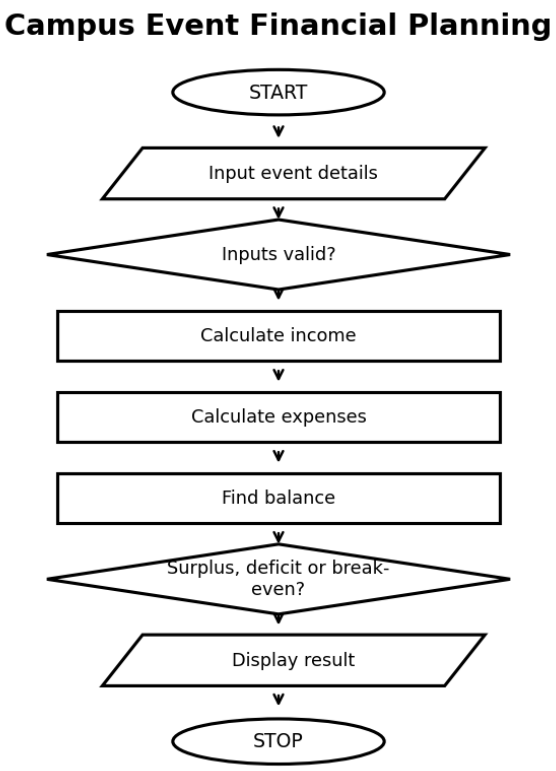
5.Calculate total expense = fixed cost + ₹200 per attendee.

6.Calculate balance = income − expense.

7.Display surplus or deficit and the amount.

8.Stop.

**FLOWCHART:**




**PSEUDOCODE:**

```
START
INPUT attendees, ticket_price, sponsorship, fixed_cost
IF any input is negative: DISPLAY "Invalid input"
ELSE:
   ticket_income = attendees * ticket_price
   income = ticket_income + sponsorship
   expense = fixed_cost + attendees * 200
   balance = income - expense
   IF balance > 0: status = "Surplus"
   ELSE IF balance < 0: status = "Deficit"
```

ELSE: status = "Break-even"

**CODE:**

```
budget = float(input("Enter event budget: "))
food = float(input("Enter food expenses: "))
decoration = float(input("Enter decoration expenses: "))
other = float(input("Enter other expenses: "))
total = food + decoration + other
remaining = budget - total
print("Total Budget: Rs.", budget)
print("Total Expenses: Rs.", total)
print("Remaining Amount: Rs.", remaining)
if remaining > 0:
    print("Budget is sufficient")
elif remaining < 0:
    print("Budget exceeded")
else:
    print("Budget fully used")
```

**OUTPUT:**

```
Enter event budget: 100000
Enter food expenses: 20000
Enter decoration expenses: 40000
Enter other expenses: 20000
Total Budget: Rs. 100000.0
Total Expenses: Rs. 80000.0
Remaining Amount: Rs. 20000.0
Budget is sufficient
```

**TESTCASES:**

## 8.Examination Result and Grace-Mark Validation

**ALGORITHM:**

1.Start.

2.Read marks in three subjects and the grace marks requested.

3.Validate marks are between 0 and 100 and grace marks are non-negative.

4.If a subject mark is below 35, calculate the shortage to reach 35.

5.If total shortage is within available grace marks, add the needed marks to the deficient subject(s).

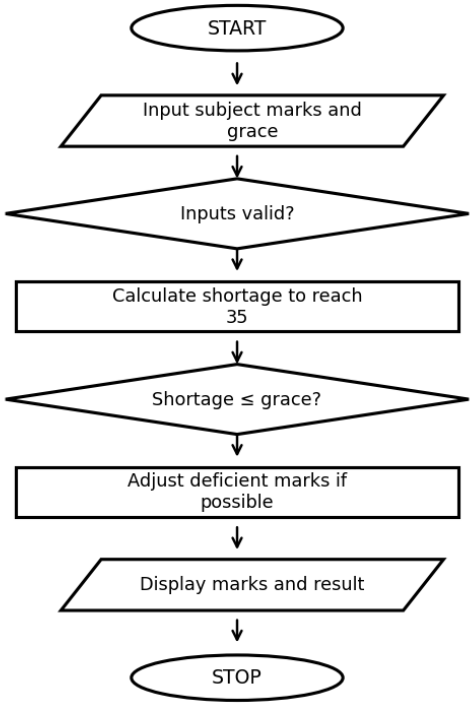
6.Classify the student as Pass if all subjects are at least 35 after adjustment; otherwise Fail.

7.Display adjusted marks and result.

8.Stop.

**FLOWCHART:**

## Examination Result and Grace-Mark Validation




**PSEUDOCODE:**

```
START
INPUT m1, m2, m3, grace
IF any mark outside 0..100 OR grace < 0: DISPLAY "Invalid input"
ELSE:
  shortage = sum(35 - mark for each mark < 35)
  IF shortage <= grace:
    add required shortage to deficient marks
    result = "Pass"
  ELSE:
    result = "Fail"
  DISPLAY adjusted marks, result
STOP
```

**CODE:**

```python
marks = int(input("Enter your marks: "))
if marks >= 35:
    print("Result: Pass")
elif marks >= 30:
    marks = marks + 5
    if marks >= 35:
        print("Result: Pass with grace marks")
    else:
        print("Result: Fail")
else:
    print("Result: Fail")

print("Final Marks:", marks)
```

**OUTPUT:**

```
Enter your marks: 90
Result: Pass
Final Marks: 90
```

**TESTCASES:**

| Test Case | Input Marks | Final Marks | Expected Output |
|---|---|---|---|
| TC1 | 40 | 40 | Pass |
| TC2 | 35 | 35 | Pass |
| TC3 | 32 | 37 | Pass with grace marks |
| TC4 | 30 | 35 | Pass with grace marks |
| TC5 | 29 | 29 | Fail |
| TC6 | 20 | 20 | Fail |
| TC7 | 0 | 0 | Fail |

## 9.Attendance recovery computation

**ALGORITHM:**

1.Start.

2.Read total classes held, classes attended, and required attendance percentage.

3.Validate total classes is positive and attended classes are within 0 to total.

4.Calculate current attendance percentage.

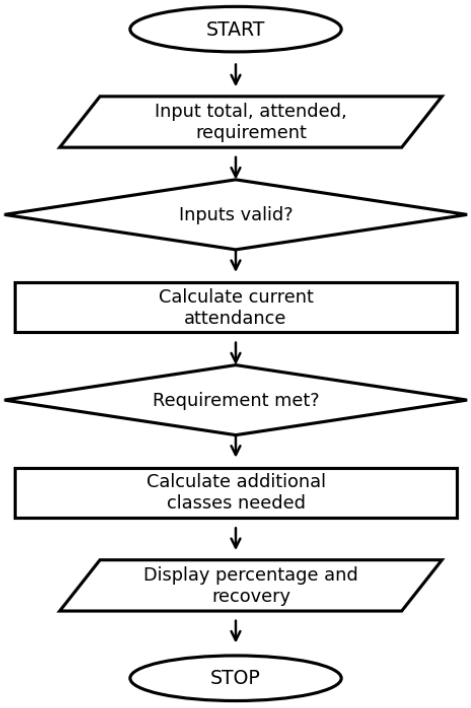
5.If attendance meets the requirement, display that no recovery is needed.

6.Otherwise calculate the minimum additional consecutive classes required, assuming all future classes are attended, to reach the requirement.

7.Display current percentage and classes needed.

8.Stop.

**FLOWCHART:**




**PSEUDOCODE:**

```
START
INPUT total, attended, required_percent
IF total <= 0 OR attended < 0 OR attended > total OR required_percent <= 0 OR required_percent > 100:
  DISPLAY "Invalid input"
ELSE:
  current = attended / total * 100
  IF current >= required_percent:
    DISPLAY "Requirement met; 0 classes needed"
  ELSE:
    needed = 0
    WHILE (attended + needed) / (total + needed) * 100 < required_percent:
      needed = needed + 1
    DISPLAY current, needed
STOP
```

**CODE:**

```python
total_classes = int(input("Enter total classes: "))
attended = int(input("Enter classes attended: "))
attendance = (attended / total_classes) * 100
print("Attendance Percentage:", attendance)
if attendance >= 75:
    print("Attendance is sufficient")
else:
    required = (0.75 * total_classes - attended) / 0.25
    print("Attendance is below 75%")
    print("More classes to attend:", int(required) + (required % 1 > 0))
```

**OUTPUT:**

```
Enter total classes: 6
Enter classes attended: 5
Attendance Percentage: 83.33333333333334
Attendance is sufficient
```

**TESTCASES:**

| Test Case | Total Classes | Classes Attended | Attendance (%) | Expected Output |
|---|---|---|---|---|
| TC1 | 100 | 80 | 80% | Attendance is sufficient |
| TC2 | 100 | 75 | 75% | Attendance is sufficient |
| TC3 | 100 | 60 | 60% | Below 75%; attend 60 more classes |
| TC4 | 80 | 40 | 50% | Below 75%; attend 80 more classes |
| TC5 | 50 | 45 | 90% | Attendance is sufficient |
| TC6 | 40 | 30 | 75% | Attendance is sufficient |
| TC7 | 20 | 10 | 50% | Below 75%; attend 20 more classes |

## 10.Advanced Numeric Pattern Generator

**ALGORITHM:**

1Start.

2.Read number of rows n.

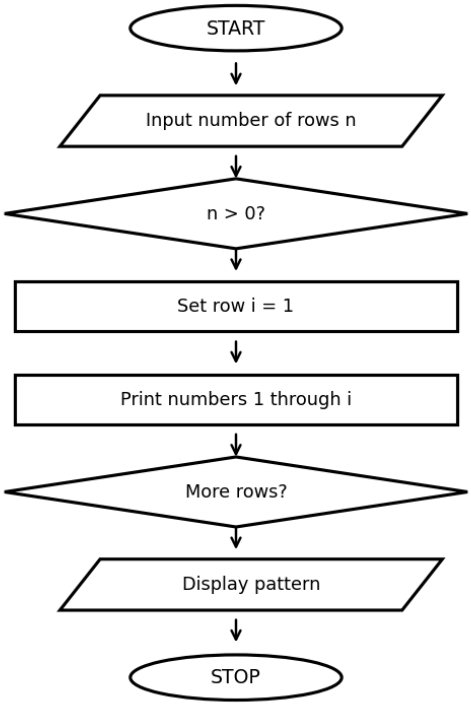
3.Validate that n is positive.

4.For each row i from 1 to n, print numbers from 1 to i in ascending order.

5.Move to the next line after each row.

6.Stop.

**FLOWCHART:**



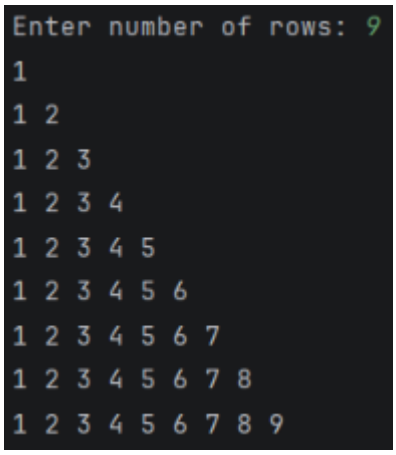

**PSEUDOCODE:**

START
INPUT n
IF n <= 0: DISPLAY "Enter a positive number"
ELSE:
  FOR i = 1 TO n:
    FOR j = 1 TO i:
      DISPLAY j without newline
    DISPLAY newline
STOP

**CODE:**

```
print("Numeric Pattern Generator")
n = int(input("Enter number of rows: "))
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

**OUTPUT:**




**TESTCASES:**

| Test Case | Input (Number of rows) | Expected Output |
|---|---|---|
| 1 | 1 | 1 |
| 2 | 2 | 1<br>1 2 |
| 3 | 3 | 1<br>1 2<br>1 2 3 |
| 4 | 4 | 1<br>1 2<br>1 2 3<br>1 2 3 4 |
| 5 | 5 | 1<br>1 2<br>1 2 3<br>1 2 3 4<br>1 2 3 4 5 |